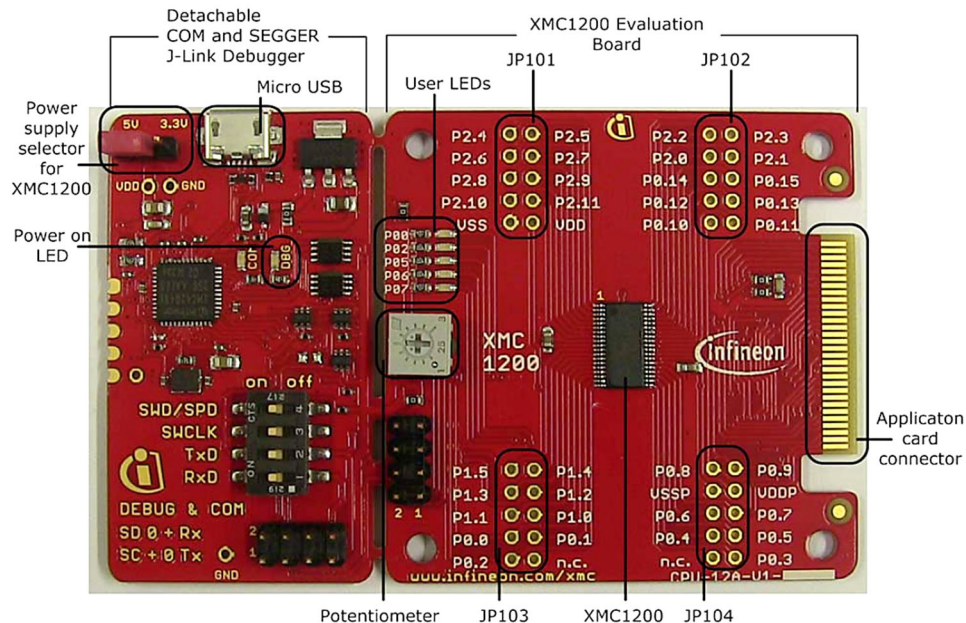


XMC1200 Boot Kit – Getting Started

1 Einleitung

Dieses Übungsblatt ist als Einstieg in die Programmierung von ARM-basierenden Mikrocontrollern gedacht. Dabei wird ein Evaluierungskit XMC1200 von der deutschen Firma Infineon AG verwendet, mit einem auf dem ARM-Cortex M0 basierenden Infineon Mikrocontroller XMC1200-T038F0200:



(From IFX XMC1200_Boot_Kit_Getting_Started)

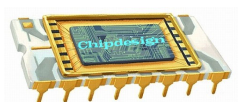
Die detailliert geführte Übung soll abwechselnd die Muster „10101“ und „01010“ auf den fünf LEDs des Evaluierungsboards anzeigen.

Die Programmierung des Controllers erfolgt mit der kostenlosen Software DAVE™ 4 von Infineon (Download <http://www.infineon.com>, Version v4.5.0). Das auf Eclipse basierende IDE (Integrated Development Environment) unterstützt die Programmierung in C, C++ und Java. Ferner ist ein Debugger, ein Library-Manager mit zahlreichen DAVE-Apps, und ein Visualisierungstool xSPY enthalten.

Detaillierte Literatur über den XMC-Controller und der IDE-Software finden Sie ebenfalls auf der Homepage vom Infineon, oder auf der Elearning-Seite:

- [1] XMC1200_Boot_Kit_Getting_started_v1-1.pdf
- [2] Board_User_Manual_XMC1200CPU_Card_R1.pdf
- [3] XMC1200_Reference_Manual_v1 0_2013_03.pdf
- [4] LED_Lighting_Application_Kit_Getting_Started_v1_2.pdf

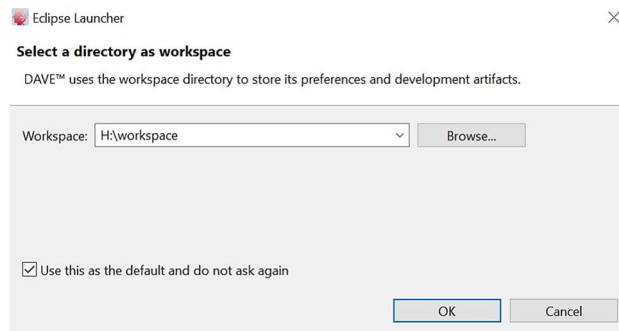
(see also ARMv6_architecture_reference_manual at <https://developer.arm.com/documentation/ddi0419>)



2 Anlegen eines DAVE™-Projektes

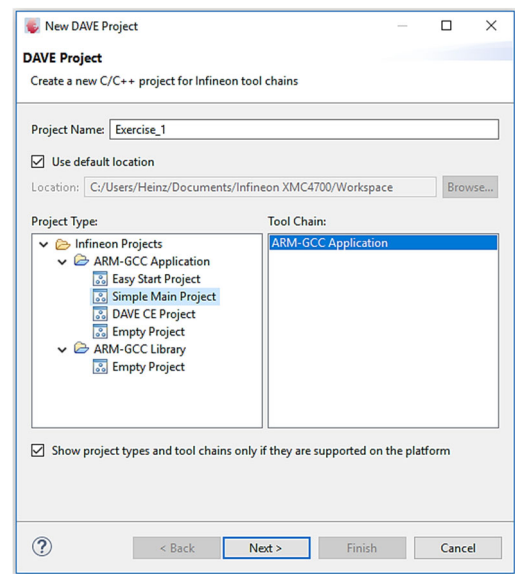
Starten Sie den Rechner des ChipLabors und melden Sie sich mit Ihrem normalen THWS-Benutzer an. Starten Sie anschließend die Eclipse-Umgebung mit DAVE-4.5.0-64Bit. Nach dem Start fragt der Workspace-Launcher nach der Position der Arbeitsumgebung.

Wichtig: Setzen Sie die Arbeitsumgebung auf Ihr lokales Laufwerk H:\workspace! Bitte hinterlassen Sie keine Daten auf den Rechnern selbst.



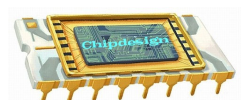
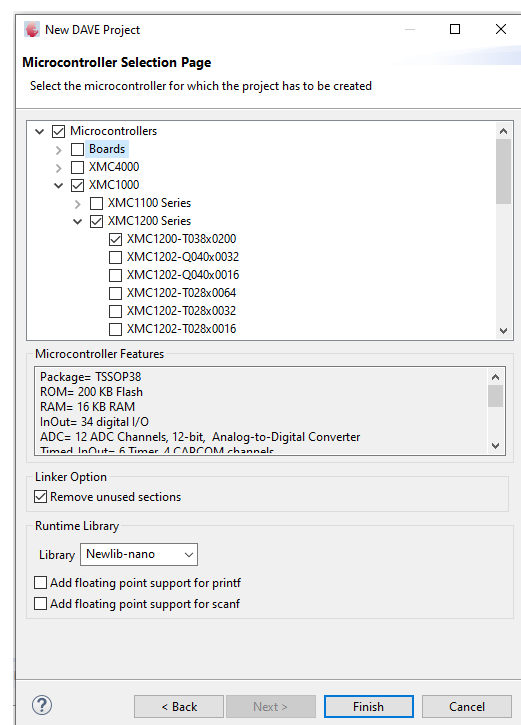
Erzeugen Sie nun ein neues, leeres Projekt mit „File → New → DAVE Project“ mit dem Name „Exercise_1“ als „Simple Main Project“:

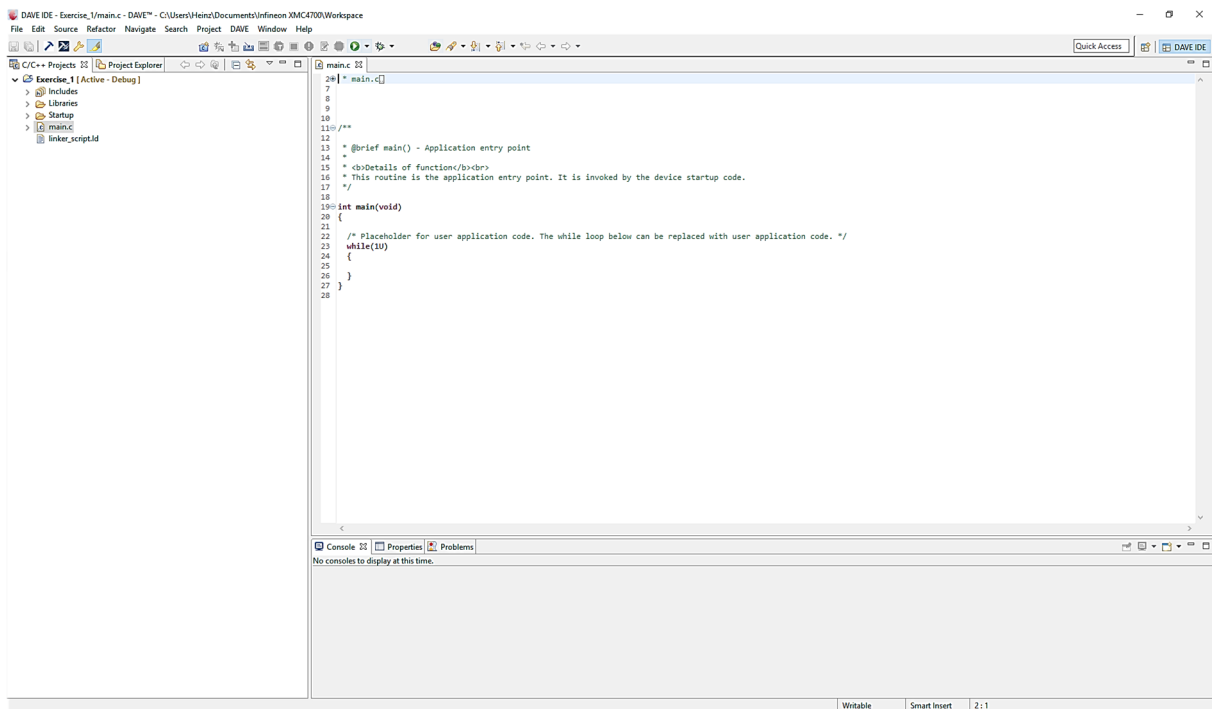
Generell: Bitte keine deutschen Umlaute im Projektnamen (oder Pfad) verwenden!



Nach Druck auf  stellen Sie den verwendeten Controller XMC1200-T038x0200 ein:

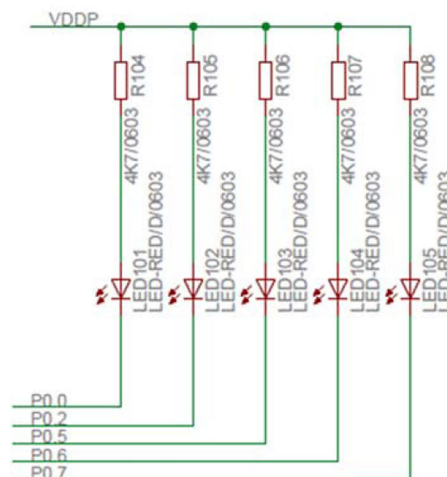
Nach  öffnen Sie im Projects-Window das Exercise_1-Projekt, und doppelklicken auf das Hauptmodel main.c. Der Bildschirm sollte folgendermaßen aussehen:





3 Konfiguration der GPIO-Boards

Der Controller verfügt insgesamt über 3 IO-Ports, mit 16 frei programmierbaren digitalen Pins an PORT0, 6 Pins an PORT1 und 12 Pins an PORT2. Die fünf LEDs des Evaluierungsboards sind dabei mit den Pins P0.0, P0.2, P0.5, P0.6 und P0.7 von PORT0 verbunden:

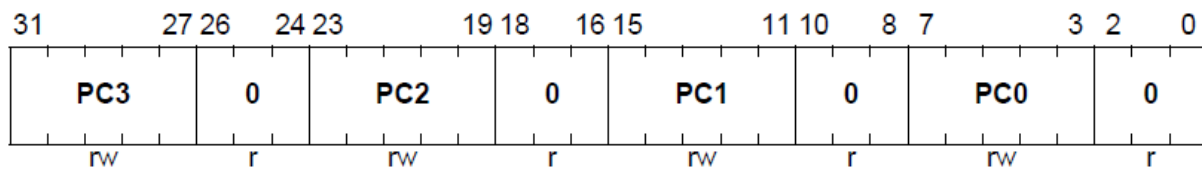


Zuerst müssen diese 5 Pins als Ausgänge definiert werden. Eine ausführliche Beschreibung hierzu finden Sie im XMC1200_Reference_Manual_*.pdf, Abschnitt „21 General Purpose IO Ports“.

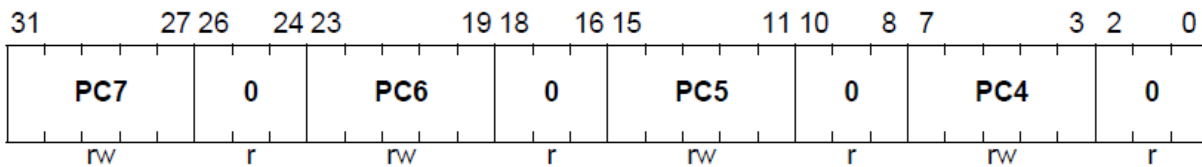
Definieren Sie die fünf Pins als push-pull Output. Dies erfolgt durch die Input/Output Control Register P0_IOCRR0 und P0_IOCRR4, in denen für jeden Pin fünf Konfigurationsbits vorgesehen sind:



Register P0_IOCR0 konfiguriert die Pins 0-3 von PORT0:



Register P0_IOCR4 konfiguriert die Pins 4-7 von PORT0:



Der Reset-Wert dieser Register ist jeweils 0x00. Für die Konfiguration der einzelnen Pins stehen folgende Möglichkeiten zur Verfügung:

Table 21-5 Standard PCx Coding¹⁾

PCx[4:0]	I/O	Output Characteristics	Selected Pull-up / Pull-down / Selected Output Function
0X000 _B	Direct Input	–	No internal pull device active
0X001 _B			Internal pull-down device active
0X010 _B			Internal pull-up device active
0X011 _B			No internal pull device active; Pn_OUTx continuously samples the input value
0X100 _B	Inverted Input	–	No internal pull device active
0X101 _B			Internal pull-down device active
0X110 _B			Internal pull-up device active
0X111 _B			No internal pull device active; Pn_OUTx continuously samples the input value

Table 21-5 Standard PCx Coding¹⁾ (cont'd)

PCx[4:0]	I/O	Output Characteristics	Selected Pull-up / Pull-down / Selected Output Function
10000 _B	Output (Direct Input)	Push-pull	General-purpose output
10001 _B			Alternate output function 1
10010 _B			Alternate output function 2
10011 _B			Alternate output function 3
10100 _B			Alternate output function 4
10101 _B			Alternate output function 5
10110 _B			Alternate output function 6
10111 _B			Alternate output function 7
11000 _B		Open-drain	General-purpose output
11001 _B			Alternate output function 1
11010 _B			Alternate output function 2
11011 _B			Alternate output function 3
11100 _B			Alternate output function 4
11101 _B			Alternate output function 5
11110 _B			Alternate output function 6
11111 _B			Alternate output function 7

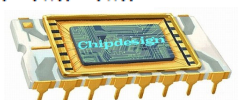
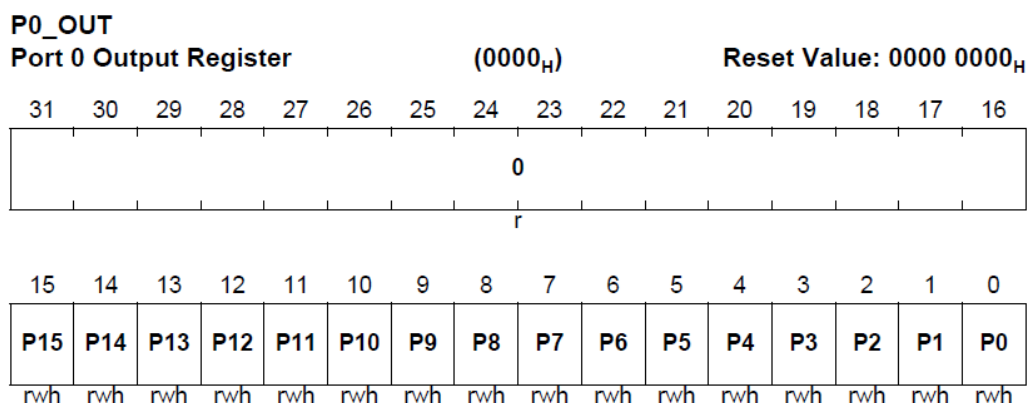
1) For the analog and digital input port P2.2 - P2.9, the combinations with PCx[4]=1_B is reserved.

Frage: Welchen (hexadezimalen) Wert müssen Sie in die Register P0_IOCR0 und P0_IOCR4 schreiben, damit die LED-Pins P0.0, P0.2, P0.5, P0.6 und P0.7 als Ausgang definiert sind?

P0_IOCR0 = _____

P0_IOCR4 = _____

Nachdem die Port-Funktion korrekt gesetzt ist, können die Ausgangswerte mit Hilfe des Ports P0_OUT gesetzt werden:



4 Verwendung der Register in C/C++

Die Steuerregister der Peripherie des Prozessors befinden sich ab der Adresse 0x4000 0000. Die einzelnen Registeradressen lassen sich im Datenblatt finden. Die DAVE-Umgebung stellt bereits eine Reihe von Include-Files zur Verfügung, welche sämtliche Register des gewählten Prozessors vordefinieren. Diese werden automatisch am Anfang von main.c über das File xmc_common.h eingebunden:

```
#include "xmc_common.h"
```

Wenn Sie mit der rechten Maustaste auf den Filenamen gehen, können Sie mit „Open Declaration“ das entsprechende File öffnen. xmc_common.h lädt das File xmc_device.h, in dem die Include-Files für die verschiedenen Prozessortypen unterschieden werden. Öffnen Sie dieses, und finden Sie die Stelle, an der das Prozessor-spezifische File XMC1200.h inkludiert wird:

```
#elif UC_SERIES == XMC12
#include "XMC1200.h"
#define CLOCK_GATING_SUPPORTED
```

Öffnen Sie auch dieses File, und suchen Sie die Definition von PORT0:

```
/* ===== */
/* ===== PORT0 ===== */
/* ===== */
typedef struct {
    __IO uint32_t OUT;          /*!< (@ 0x40040000) PORT0 Structure */
    __IO uint32_t OMR;          /*!< (@ 0x40040000) Port 0 Output Register */
    __IO uint32_t RESERVED0[2]; /*!< (@ 0x40040004) Port 0 Output Modification Register */
    __IO uint32_t IOCR0;        /*!< (@ 0x40040010) Port 0 Input/Output Control Register 0 */
    __IO uint32_t IOCR4;        /*!< (@ 0x40040014) Port 0 Input/Output Control Register 4 */
    __IO uint32_t IOCR8;        /*!< (@ 0x40040018) Port 0 Input/Output Control Register 8 */
    __IO uint32_t IOCR12;       /*!< (@ 0x4004001C) Port 0 Input/Output Control Register 12 */
    __IO uint32_t RESERVED1;
    __IO uint32_t IN;           /*!< (@ 0x40040024) Port 0 Input Register */
    __IO uint32_t RESERVED2[6];
    __IO uint32_t PHCR0;        /*!< (@ 0x40040040) Port 0 Hysteresis Control Register 0 */
    __IO uint32_t PHCR1;        /*!< (@ 0x40040044) Port 0 Hysteresis Control Register 1 */
    __IO uint32_t RESERVED3[6];
    __IO uint32_t PDISC;        /*!< (@ 0x40040060) Port 0 Pin Function Decision Control */
    __IO uint32_t RESERVED4[3];
    __IO uint32_t PPS;          /*!< (@ 0x40040070) Port 0 Pin Power Save Register */
    __IO uint32_t HWSEL;        /*!< (@ 0x40040074) Port 0 Pin Hardware Select Register */
} PORT0_Type;
```

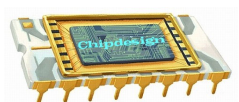
...

```
#define PORT0_BASE                0x40040000UL
```

...

```
#define PORT0                      ((PORT0_Type *) PORT0_BASE)
```

PORT0 ist somit ein Zeiger auf ein struct vom Typ PORT0_Type mit der Basisadresse 0x4004 0000. Um beispielsweise vom Register IN von PORT0 zu lesen, kann man den Dereferenzierungsoperator (*PORT0).IN verwenden, oder die abkürzende Schreibweise PORT0->IN.



Vervollständigen Sie das main.c-Programm, so dass auf den LEDs das Muster „10101“ angezeigt wird. Beachten Sie dabei, dass entsprechend dem Schaltplan eine LED leuchtet, wenn eine ‚0‘ ausgegeben wird:

```
int main(void)
{
    PORT0->IOCR0 = _____

    PORT0->IOCR4 = _____

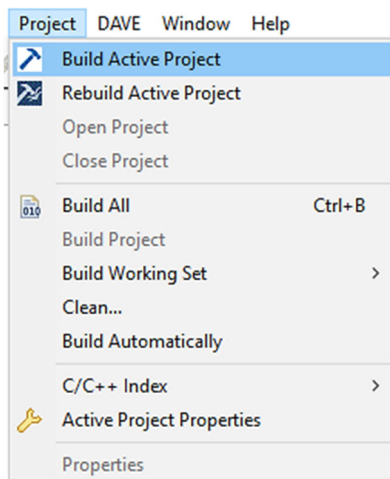
    PORT0->OUT    = _____

    while(1)
    { }
}
```

5 Kompilieren des Programms und übertragen auf das Demo-Board

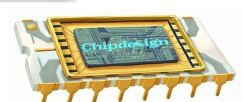
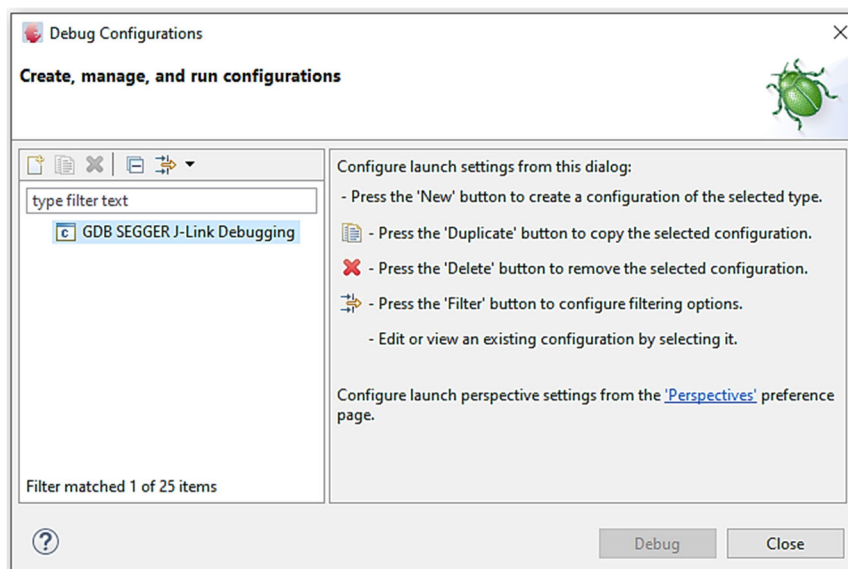
Schließen Sie entsprechend der Abbildung das Demo-Board über das USB-Kabel am PC an.

Anschließend starten Sie das Kompilieren mit Project → Build Active Project:

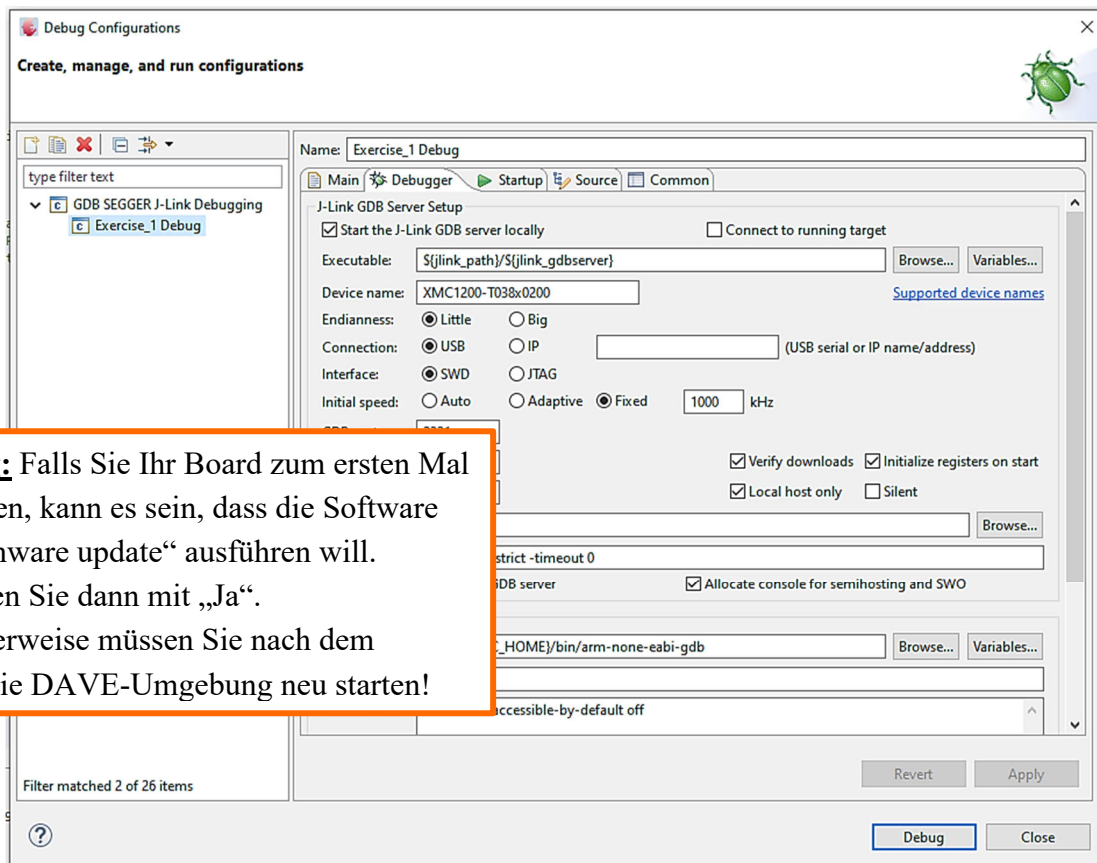


Falls der Code korrekt kompiliert hat, starten sie den Debugger mit  (F11).

Wählen Sie im anschließenden Pop-Up den GDB SEGGER J-Link Debugger durch Doppelklick aus:



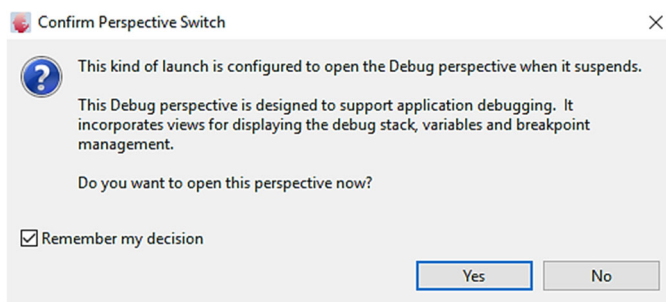
Stellen Sie sicher, dass im Reiter „Debugger“ das J-Link Executable richtig eingestellt ist:



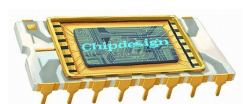
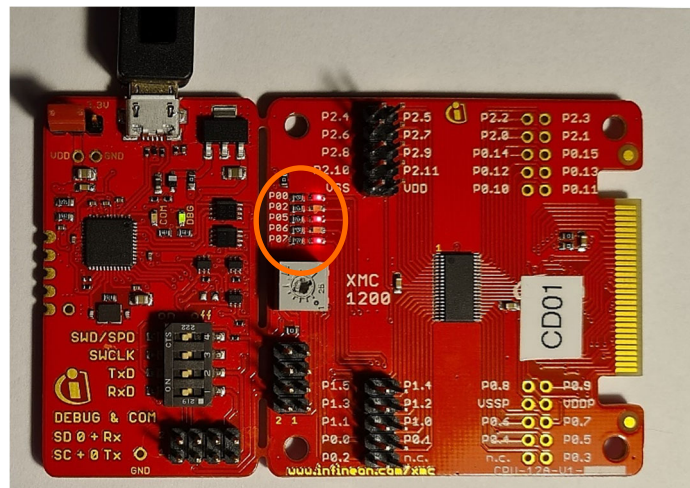
Achtung: Falls Sie Ihr Board zum ersten Mal verwenden, kann es sein, dass die Software ein „Firmware update“ ausführen will. Antworten Sie dann mit „Ja“.

Möglicherweise müssen Sie nach dem Update die DAVE-Umgebung neu starten!

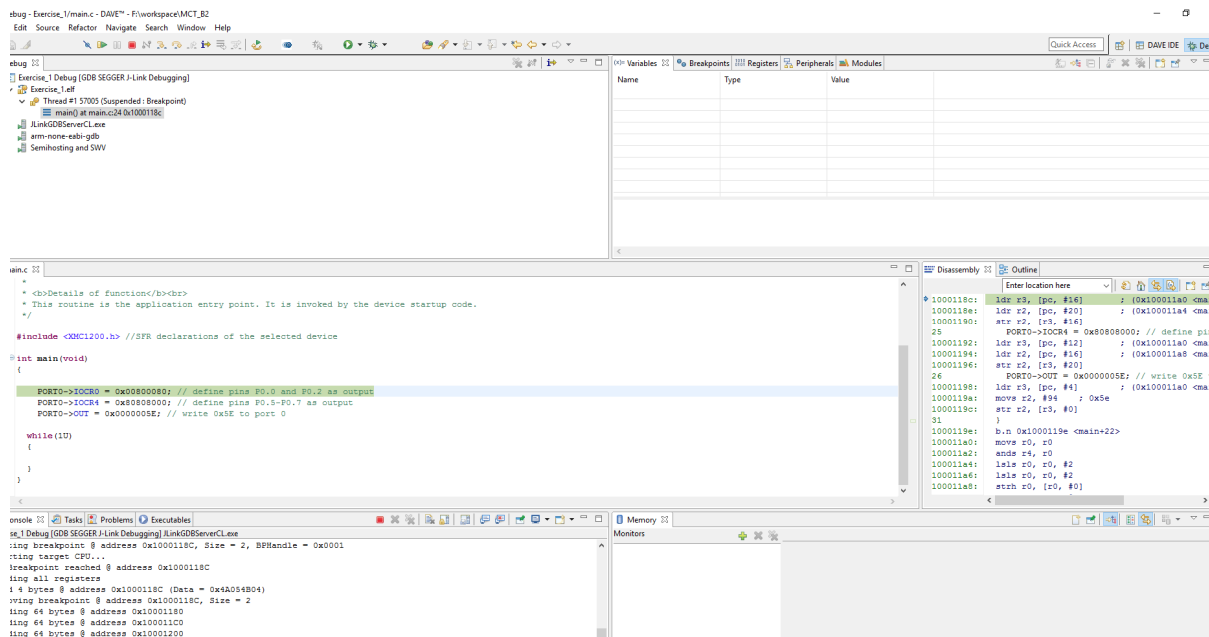
und starten mit . Das Tool wechselt dabei automatisch in die Debug-Umgebung, falls Sie die entsprechende Option aktivieren:



Starten Sie das Programm mit . Falls alles richtig ist, sollten die LED folgendermaßen aufleuchten:



Die korrekte Debug-Umgebung sollte dann folgendermaßen aussehen:



6 Betrachten des ARM Maschinencodes

Im „Disassembly“-Fenster wird der Maschinencode, den der Compiler erzeugt hat, angezeigt:

```

PORT0->IOCR0 = 0x00800080;
0x1000118c 04 4b      ldr      r3, [pc, #0x10]
0x1000118e 05 4a      ldr      r2, [pc, #0x14]
0x10001190 1a 61      str      r2, [r3, #0x10]
PORT0->IOCR4 = 0x80808000;
0x10001192 03 4b      ldr      r3, [pc, #0xc]
0x10001194 04 4a      ldr      r2, [pc, #0x10]
0x10001196 5a 61      str      r2, [r3, #0x14]
PORT0->OUT = 0x00000044;
0x10001198 01 4b      ldr      r3, [pc, #0x4]
0x1000119a 44 22      movs     r2, #0x44
0x1000119c 1a 60      str      r2, [r3, #0x0]
}
0x1000119e fe e7      b        1000119e
0x100011a0 00 00
0x100011a2 04 40
0x100011a4 80 00
0x100011a6 80 00
0x100011a8 00 80
0x100011aa 80 80

```

Versuchen Sie, den Assembler-Code zu verstehen. Verwenden Sie dazu die entsprechende Information aus dem Vorlesungsskript. Notieren Sie zu jeder Assembler-Zeile die genaue Bedeutung.

